

APPENDIX

B

Lab Programs

Program 1 Write a shell script to print all the prime numbers between M to N ($M < N$).

```
#!/bin/bash
```

```
read -p "Enter M: " M
```

```
read -p "Enter N: " N
```

```
echo "Prime Numbers between $M and $N are : "
```

```
for ((n=M; n<=N; n++)); do
```

```
    prime=1
```

```
    for ((i=2; i*i<=n; i++)); do
```

```
        if (( n % i == 0 )); then
```

```
            prime=0
```

```
            break
```

```
        fi
```

```
    done
```

```
    (( n > 1 && prime )) && echo -n "$n "
```

```
done
```

```
echo ""
```

Output

```
Enter M: 10
```

```
Enter N: 50
```

```
Prime Numbers between 10 and 50 are :
```

```
11 13 17 19 23 29 31 37 41 43 47
```


Program 2

Write a shell script to reverse a given number and check whether it is a palindrome.

```
#!/bin/bash
# Prompt the user to enter a number
read -p "Enter a number: " num

# Store the original number
original_num=$num

# Reverse the number
reverse=0
while (( num > 0 )); do
    digit=$(( num % 10 )) # Get the last digit
    reverse=$(( reverse * 10 + digit )) # Add the digit to reverse
    num=$(( num / 10 )) # Remove the last digit
done

# Check if the original number is equal to the reversed number
if (( original_num == reverse )); then
    echo "The number $original_num is a palindrome."
else
    echo "The number $original_num is not a palindrome."
fi
```

Output

Enter a number: 1234
The number 1234 is not a palindrome.

Enter a number: 12321
The number 12321 is a palindrome.

Program 3(a) Write a shell script to find the sum of digits of a given number using loops

```
#!/bin/bash

# Prompt the user to enter a number
read -p "Enter a number: " num

# Initialize the sum variable
sum=0

# Calculate the sum of digits using a loop
while (( num > 0 )); do
    digit=$(( num % 10 )) # Get the last digit
    sum=$(( sum + digit )) # Add the digit to the sum
    num=$(( num / 10 )) # Remove the last digit
done

# Output the result
echo "The sum of digits is: $sum"
```

Output

Enter a number: 1234

The sum of digits is: 10

Program 3 (b)

Write a shell script to find the sum of digits of a given number without using loops.

```
#!/bin/bash

# Function to calculate the sum of digits using recursion
sum_digits() {
    num=$1
    if [ $num -lt 10 ]; then
        echo $num
    else
        last_digit=$((num % 10))
        remaining_number=$((num / 10))
        sum=$((last_digit + $(sum_digits $remaining_number)))
        echo $sum
    fi
}

# Read the number
read -p "Enter a number: " number

# Call the sum_digits function and display the result
sum=$(sum_digits $number)
echo "The sum of the digits is: $sum"
```

Output

Enter a number: 12345

The sum of the digits is: 15

Program 4 Write a shell script to implement 10 Linux commands using case.

```
#!/bin/bash

# prompt the user for input
echo "Choose a Linux command (1-10):"
echo "1. List files"
echo "2. Show current directory"
echo "3. Display date and time"
echo "4. Show system uptime"
echo "5. Show disk usage"
echo "6. Show memory usage"
echo "7. Show the last logged-in user"
echo "8. Show running processes"
echo "9. Show system information"
echo "10. Display hostname"

read -p "Enter your choice (1-10): " choice

# Implement commands using case
case $choice in
    1) ls ;;
    2) pwd ;;
    3) date ;;
    4) uptime ;;
    5) df ;;
    6) free ;;
    7) whoami ;;
    8) ps ;;
    9) uname -a ;;
    10) hostname ;;
    *) echo "Invalid choice. Please select a number between 1 and 10." ;;
esac
```

Output

```
Choose a Linux command (1-10):
1. List files
2. Show current directory
3. Display date and time
```


4. Show system uptime
5. Show disk usage
6. Show memory usage
7. Show the last logged-in user
8. Show running processes
9. Show system information
10. Display hostname

Enter your choice (1-10): 10

Srikanth

Program 5

Write a Shell script that displays list of all the files in the current directory to which the user has read, write and execute permissions.

```
#!/bin/bash
```

```
# List files with read, write, and execute permissions for the user
```

```
for file in *; do
```

```
    if [ -f "$file" ] && [ -r "$file" ] && [ -w "$file" ] && [ -x "$file" ]; then
```

```
        echo "$file"
```

```
    fi
```

```
done
```

Output

MyFirstApp.jks

MyKeystore.jks

NTUSER.DAT

NTUSER.DAT{97544b6e-e4ce-11ef-bae8-bd8fb8fb9a02}.TM.blf

file1.txt

file2.txt

ntuser.dat.LOG1

ntuser.dat.LOG2

ntuser.ini

script.sh

Program 6 Write a shell script to copy a file within current directory.

```
#!/bin/bash
```

```
# Prompt user for the file name to copy
```

```
read -p "Enter the file name to copy: " filename
```

```
# Check if the file exists
```

```
if [ -f "$filename" ]; then
```

```
    # Prompt for the new file name
```

```
    read -p "Enter the new file name: " new_filename
```

```
    # Copy the file to the new name
```

```
    cp "$filename" "$new_filename"
```

```
    echo "File copied successfully to $new_filename"
```

```
else
```

```
    echo "File does not exist."
```

```
fi
```

Output

```
Enter the file name to copy: file1.txt
```

```
Enter the new file name: mynewfile.txt
```

```
File copied successfully to mynewfile.txt
```


doesn't exist.

Program 7 Write a shell script to copy file between two directories.

```
#!/bin/bash

# Prompt the user to enter the source file and the destination directory
read -p "Enter the source file path: " source_file
read -p "Enter the destination directory path: " dest_dir

# Check if the source file exists
if [ -f "$source_file" ]; then
    # Check if the destination directory exists
    if [ -d "$dest_dir" ]; then
        # Copy the file to the destination directory
        cp "$source_file" "$dest_dir"
        echo "File copied successfully to $dest_dir"
    else
        echo "Destination directory does not exist."
    fi
else
    echo "Source file does not exist."
fi
```

Output

Enter the source file path: file1.txt

Enter the destination directory path: /c/Users/chett/Documents

File copied successfully to /c/Users/chett/Documents

Program 8

Write a Shell script to create two data files and compare them to display unique and common entries.

```
#!/bin/bash

# Create two data files with sample data
echo -e "apple\nbanana\ncherry\norange\n" > file1.txt
echo -e "banana\ncherry\ngrapes\nmelon\n" > file2.txt

# Sort data files
sort file1.txt -o file1.txt
sort file2.txt -o file2.txt

# Display common entries
echo "Common entries between file1.txt and file2.txt:"
comm -12 file1.txt file2.txt

# Display unique entries in file1.txt
echo "Unique entries in file1.txt:"
comm -23 file1.txt file2.txt

# Display unique entries in file2.txt
echo "Unique entries in file2.txt:"
comm -13 file1.txt file2.txt
```

Output

Common entries between file1.txt and file2.txt:

banana

cherry

Unique entries in file1.txt:

apple

orange

Unique entries in file2.txt:

grapes

melon

Sample data (list of fruits)

Program 9 Write a shell script to count the number of vowels in a string.

```
#!/bin/bash

# Prompt the user to enter a string
read -p "Enter a string: " input_string

# Convert the string to lowercase, then extract vowels and count them
vowel_count=$(echo "$input_string" | tr -cd 'aeiouAEIOU' | wc -c)

# Display the result
echo "The number of vowels in the string is: $vowel_count"
```

Output

```
Enter a string: Skyward Publishers
The number of vowels in the string is: 4
```


Program 10

Write a shell script to convert uppercase characters to lowercase and vice versa.

```
#!/bin/bash

# Prompt the user to enter a string
read -p "Enter a string: " input_string

# Convert uppercase to lowercase and lowercase to uppercase
str=$(echo "$input_string" | tr 'a-zA-Z' 'A-Za-z')

# Display the result
echo "Converted string: $str"
```


Output
Enter a string: LINUX Programming Converted string: linux pROGRAMMING

Program 11

Write a shell script to accept a word and perform pattern matching in a given file.

```
#!/bin/bash

# Prompt the user to enter a word to search for
read -p "Enter the word to search: " word

# Prompt the user to enter the filename
read -p "Enter the file name: " filename

# Check if the file exists
if [ -f "$filename" ]; then
    # Perform pattern matching using grep
    echo "Searching for '$word' in $filename..."
    grep -n "$word" "$filename"
else
    echo "File does not exist."
fi
```

Output

```
Enter the word to search: boy
Enter the file name: file.txt
Searching for 'boy' in file.txt...
1:Srikanth is a good boy.
2:Amar is an intelligent boy.
4:Kishore is a bad boy.
```


Program 12 Write a shell script to find factorial of a number.

```
#!/bin/bash

# Prompt the user to enter a number
read -p "Enter a number: " num

# Initialize the factorial variable
factorial=1

# Check if the number is negative
if [ $num -lt 0 ]; then
    echo "Factorial is not defined for negative numbers."
else
    # Calculate the factorial
    for (( i=1; i<=num; i++ )); do
        factorial=$(( factorial * i ))
    done

    # Display the result
    echo "The factorial of $num is: $factorial"
fi
```

Output

```
Enter a number: 5
The factorial of 5 is: 120
```


Program 13

Write a Menu driven program to demonstrate zombie process and orphan process.

Appendix B : Lab Programs **B.13**

```
#!/bin/bash

# Function to create a Zombie Process
create_zombie() {
    echo "Creating Child Process..."

    # Create a child process that exits after 3 seconds
    (
        sleep 3
        echo "Child Process has completed execution."
        echo "I am a Zombie while the parent sleeps."
    ) &

    # Inform user that the zombie process should be visible
    echo "The parent process is now sleeping for 30 seconds."

    # Parent sleeps, allowing the child to finish and become a zombie
    sleep 20

    # Parent finishes execution
    echo "Parent process has finished. I'm back!"
    echo "Child is no longer a Zombie; system has cleaned it up."
}

# Function to create an Orphan Process
create_orphan() {
    echo "Creating a Child Process..."

    # Start a child process in the background
    (
        echo "Child Process created."
        sleep 10 # Child process sleeps for 10 seconds
        echo "Child Process completed execution. Now I will exit."
    ) &

    # Parent process exits immediately
    echo "Parent exiting early; child becomes an orphan."
    exit
}
```



```

# Menu-driven program
while true; do
    echo -e "\nZombie and Orphan Process Demonstration"
    echo "1. Create Zombie Process"
    echo "2. Create Orphan Process"
    echo "3. Exit"
    read -p "Enter your choice: " choice

    case $choice in
        1) create_zombie ;;
        2) create_orphan ;;
        3) echo "Exiting..."; exit ;;
        *) echo "Invalid choice! Try again." ;;
    esac
done

```

Output

Zombie and Orphan Process Demonstration 1. Create Zombie Process 2. Create Orphan Process 3. Exit Enter your choice: 1 Creating Child Process... The parent process is now sleeping for 30 seconds. Child Process has completed execution. I am a Zombie while the parent sleeps. Parent process has finished. I'm back! Child is no longer a Zombie; system has cleaned it up.
--

Zombie and Orphan Process Demonstration 1. Create Zombie Process 2. Create Orphan Process 3. Exit Enter your choice: 2 Creating a Child Process... Parent exiting early; child becomes an orphan. Child Process created. Child Process completed execution. Now I will exit.
